

A Chair-Readable Derivation of the Fixed-Branch Squared Tensor-Train Gradient

BayesFilter Zhao–Cui Gradient Note

2026-06-01

Abstract

This note explains the analytical gradient used by the fixed-branch squared tensor-train filtering construction inspired by Zhao and Cui. The purpose is not to introduce a new filtering method. The purpose is to make one derivative claim understandable: after all approximation choices are frozen, the derivative computed by the backward calculation is the derivative of the same approximate normalizing-constant scalar computed by the forward calculation. The exposition starts with scalar normalizing constants, then squared approximations, then a two-coordinate tensor-train warmup, then fixed linear solves, and only then the full tensor-train filtering derivative.

Contents

1	Motivation For The Panel Chair	2
2	Warmup 1: A Normalizing Constant	3
3	Warmup 2: A Squared Approximation	3
4	Warmup 3: Two Coordinates, Rank One	4
5	Warmup 4: Two Coordinates, Rank R	5
6	Warmup 5: A Fixed Linear Solve	6
7	The Fixed-Branch Forward Pass	7
8	The Fixed-Branch Derivative Pass	8
8.1	Target And Square-Root Target	8
8.2	Design Matrix And Core Solve	9
8.3	Mass Contraction And Normalizer	11
8.4	Carried Filter And Next Time Step	11
9	Proposition 1: The Fixed-Branch Recursion Is A Normalized Approximate Filter	12
10	Proposition 2: The Fixed-Branch Derivative Differentiates The Same Scalar	13
11	What This Does Not Prove	14

12 Finite-Difference Test	15
13 Minimal Runnable Example	16
14 Closing Summary	17

1 Motivation For The Panel Chair

The filtering algorithm produces, at each time t , an approximate normalizing constant

$$\hat{Z}_t(\beta) > 0. \quad (1)$$

The parameter β is the model parameter at which we evaluate the approximate likelihood. The total approximate log likelihood is

$$\hat{\ell}_T(\beta) = \sum_{t=1}^T \log \hat{Z}_t(\beta). \quad (2)$$

For a chemist, the closest familiar object is a partition function. A partition function is an integral of a nonnegative weight. Here $\hat{Z}_t(\beta)$ is also an integral of a nonnegative weight:

$$\hat{Z}_t(\beta) = \int \hat{q}_t(z; \beta) dz. \quad (3)$$

The variable z is a numerical coordinate, usually a transformed coordinate on a fixed box. The function $\hat{q}_t$ is not the exact Bayesian density. It is the squared tensor-train approximation used by the numerical filter.

The panel cares about the gradient because parameter inference algorithms need to know how the approximate log likelihood changes when β changes:

$$\nabla_{\beta} \hat{\ell}_T(\beta). \quad (4)$$

The delicate point is not the derivative of log. The delicate point is that $\hat{Z}_t$ is produced by a complicated approximation algorithm. If the approximation algorithm changes its grid, ranks, pivots, or scaling shifts when β changes, then the two likelihood evaluations may not be evaluations of the same mathematical formula. A classical analytical derivative can only differentiate one declared formula.

That is why this note uses a fixed branch.

Fixed branch. Before differentiating, freeze the computational choices: coordinate domain, fitting points, basis functions, tensor-train ranks, sweep order, ridge parameter, defensive density, stabilizing shifts, and the rule for constructing every fitted core. Do not freeze the fitted core values themselves. Those core values are outputs of the fixed fitting equations, so they remain functions of β . Then differentiate the scalar defined by the saved choices and by the core values obtained from the same saved fitting equations.

The goal is therefore:

$$\text{differentiate the scalar actually computed by the saved forward pass.} \quad (5)$$

The goal is not:

$$\text{differentiate every possible adaptive version of the algorithm.} \quad (6)$$

2 Warmup 1: A Normalizing Constant

Start with the simplest possible object. Let

$$Z(\beta) = \int_{\Omega} q(z; \beta) \, dz, \quad q(z; \beta) \geq 0, \quad 0 < Z(\beta) < \infty. \quad (7)$$

The normalized density is

$$p(z; \beta) = \frac{q(z; \beta)}{Z(\beta)}. \quad (8)$$

The log normalizing constant is

$$L(\beta) = \log Z(\beta). \quad (9)$$

Assume that $q(z; \beta)$ is differentiable in β , and that we may move the derivative inside the integral:

$$\frac{\partial}{\partial \beta} \int_{\Omega} q(z; \beta) \, dz = \int_{\Omega} \frac{\partial q(z; \beta)}{\partial \beta} \, dz. \quad (10)$$

A sufficient condition is that, near the parameter value of interest, the absolute derivative is bounded by an integrable function:

$$\left| \frac{\partial q(z; \beta)}{\partial \beta} \right| \leq m(z), \quad \int_{\Omega} m(z) \, dz < \infty. \quad (11)$$

Then the derivative of the log normalizer is

$$\begin{aligned} \frac{\partial L(\beta)}{\partial \beta} &= \frac{\partial}{\partial \beta} \log Z(\beta) \\ &= \frac{1}{Z(\beta)} \frac{\partial Z(\beta)}{\partial \beta} \\ &= \frac{1}{Z(\beta)} \int_{\Omega} \frac{\partial q(z; \beta)}{\partial \beta} \, dz. \end{aligned} \quad (12)$$

This is the first idea in the whole gradient. Everything later is only a way to compute the numerator and denominator in (12) for the tensor-train approximation.

For the sequential filter, the same formula is used at every time:

$$\frac{\partial \hat{\ell}_T(\beta)}{\partial \beta} = \sum_{t=1}^T \frac{1}{\hat{Z}_t(\beta)} \frac{\partial \hat{Z}_t(\beta)}{\partial \beta}. \quad (13)$$

3 Warmup 2: A Squared Approximation

The fixed-branch squared tensor-train approximation has the form

$$\hat{q}(z; \beta) = e^{-c} \phi(z; \beta)^2 + \tau \lambda(z). \quad (14)$$

Here:

$$\begin{array}{ll} c & \text{is a frozen stabilizing shift,} \\ \tau > 0 & \text{is a frozen defensive mass,} \\ \lambda(z) \geq 0 & \text{is a frozen reference density with } \int \lambda = 1, \\ \phi(z; \beta) & \text{is the fitted square-root function.} \end{array} \quad (15)$$

Because c, τ, λ are frozen, the only object in (14) that changes with β is ϕ . Differentiating (14) gives

$$\begin{aligned}\dot{\hat{q}}(z; \beta) &= \frac{\partial}{\partial \beta} [e^{-c} \phi(z; \beta)^2 + \tau \lambda(z)] \\ &= e^{-c} \frac{\partial}{\partial \beta} \phi(z; \beta)^2 + \frac{\partial}{\partial \beta} \tau \lambda(z) \\ &= 2e^{-c} \phi(z; \beta) \dot{\phi}(z; \beta).\end{aligned}\tag{16}$$

The dot means $\partial/\partial\beta$.

The corresponding normalizer is

$$\begin{aligned}\hat{Z}(\beta) &= \int \hat{q}(z; \beta) \, dz \\ &= e^{-c} \int \phi(z; \beta)^2 \, dz + \tau \int \lambda(z) \, dz \\ &= e^{-c} \int \phi(z; \beta)^2 \, dz + \tau.\end{aligned}\tag{17}$$

Using (16), its derivative is

$$\begin{aligned}\dot{\hat{Z}}(\beta) &= \int \dot{\hat{q}}(z; \beta) \, dz \\ &= 2e^{-c} \int \phi(z; \beta) \dot{\phi}(z; \beta) \, dz.\end{aligned}\tag{18}$$

Substitution into (12) gives

$$\frac{\partial}{\partial \beta} \log \hat{Z}(\beta) = \frac{2e^{-c} \int \phi(z; \beta) \dot{\phi}(z; \beta) \, dz}{e^{-c} \int \phi(z; \beta)^2 \, dz + \tau}.\tag{19}$$

If c, τ , or λ changed with β , then (16) would have extra terms:

$$\dot{\hat{q}} = -\dot{c} e^{-c} \phi^2 + 2e^{-c} \phi \dot{\phi} + \dot{\tau} \lambda + \tau \dot{\lambda}.\tag{20}$$

The fixed-branch derivative does not use (20). It uses the fixed-branch formula (16). This is not a minor technicality. It is the mathematical line between differentiating one saved scalar and differentiating a changing adaptive procedure.

4 Warmup 3: Two Coordinates, Rank One

Before a full tensor train, consider a two-coordinate rank-one function:

$$\phi(z_1, z_2; \beta) = h_1(z_1; \beta) h_2(z_2; \beta).\tag{21}$$

The square integral is

$$\begin{aligned}R(\beta) &= \iint \phi(z_1, z_2; \beta)^2 \, dz_1 \, dz_2 \\ &= \iint h_1(z_1; \beta)^2 h_2(z_2; \beta)^2 \, dz_1 \, dz_2 \\ &= \left[\int h_1(z_1; \beta)^2 \, dz_1 \right] \left[\int h_2(z_2; \beta)^2 \, dz_2 \right].\end{aligned}\tag{22}$$

Define

$$R_1(\beta) = \int h_1(z_1; \beta)^2 dz_1, \quad R_2(\beta) = \int h_2(z_2; \beta)^2 dz_2. \quad (23)$$

Then

$$R(\beta) = R_1(\beta)R_2(\beta). \quad (24)$$

Differentiate by the product rule:

$$\dot{R}(\beta) = \dot{R}_1(\beta)R_2(\beta) + R_1(\beta)\dot{R}_2(\beta). \quad (25)$$

Each one-dimensional derivative is

$$\dot{R}_1(\beta) = 2 \int h_1(z_1; \beta) \dot{h}_1(z_1; \beta) dz_1, \quad (26)$$

$$\dot{R}_2(\beta) = 2 \int h_2(z_2; \beta) \dot{h}_2(z_2; \beta) dz_2. \quad (27)$$

Combining (25)–(27),

$$\begin{aligned} \dot{R}(\beta) = & 2 \left[\int h_1 \dot{h}_1 dz_1 \right] \left[\int h_2^2 dz_2 \right] \\ & + 2 \left[\int h_1^2 dz_1 \right] \left[\int h_2 \dot{h}_2 dz_2 \right]. \end{aligned} \quad (28)$$

This is the simplest version of the tensor-train mass derivative: when one factor changes, keep the other factors fixed and sum the contributions.

5 Warmup 4: Two Coordinates, Rank R

Now let

$$\phi(z_1, z_2; \beta) = \sum_{r=1}^R h_{1r}(z_1; \beta) h_{2r}(z_2; \beta). \quad (29)$$

The square integral is

$$\begin{aligned} R(\beta) = & \iint \left[\sum_{r=1}^R h_{1r}(z_1) h_{2r}(z_2) \right] \left[\sum_{s=1}^R h_{1s}(z_1) h_{2s}(z_2) \right] dz_1 dz_2 \\ = & \sum_{r=1}^R \sum_{s=1}^R \left[\int h_{1r}(z_1) h_{1s}(z_1) dz_1 \right] \left[\int h_{2r}(z_2) h_{2s}(z_2) dz_2 \right]. \end{aligned} \quad (30)$$

Define the two mass matrices

$$M_1[r, s] = \int h_{1r}(z_1) h_{1s}(z_1) dz_1, \quad (31)$$

$$M_2[r, s] = \int h_{2r}(z_2) h_{2s}(z_2) dz_2. \quad (32)$$

Then

$$R(\beta) = \sum_{r,s} M_1[r, s] M_2[r, s]. \quad (33)$$

This is the Frobenius inner product

$$R(\beta) = \langle M_1(\beta), M_2(\beta) \rangle_F. \quad (34)$$

Differentiate:

$$\dot{R} = \langle \dot{M}_1, M_2 \rangle_F + \langle M_1, \dot{M}_2 \rangle_F. \quad (35)$$

The derivative of a mass matrix entry is

$$\dot{M}_1[r, s] = \int \dot{h}_{1r}(z_1) h_{1s}(z_1) + h_{1r}(z_1) \dot{h}_{1s}(z_1) dz_1, \quad (36)$$

and similarly for M_2 . The full tensor-train mass recursion is only this calculation repeated along a chain of coordinates.

6 Warmup 5: A Fixed Linear Solve

The fitted tensor-train cores are obtained from finite-dimensional least-squares updates. One such update can be written

$$N(\beta)g(\beta) = d(\beta). \quad (37)$$

Here g is the vector of unknown core coefficients in the current update, N is the regularized normal matrix, and d is the right-hand side. Differentiate both sides:

$$\frac{\partial}{\partial \beta} \{N(\beta)g(\beta)\} = \frac{\partial d(\beta)}{\partial \beta}. \quad (38)$$

Use the product rule:

$$\dot{N}(\beta)g(\beta) + N(\beta)\dot{g}(\beta) = \dot{d}(\beta). \quad (39)$$

Move the first term to the right:

$$N(\beta)\dot{g}(\beta) = \dot{d}(\beta) - \dot{N}(\beta)g(\beta). \quad (40)$$

If $N(\beta)$ is nonsingular, then

$$\dot{g}(\beta) = N(\beta)^{-1} [\dot{d}(\beta) - \dot{N}(\beta)g(\beta)]. \quad (41)$$

In computation, one should not form the inverse. One solves the linear system (40) using the same numerical linear algebra used for the forward solve.

The ridge term is important because the normal matrix has the form

$$N = A^\top W A + \rho I, \quad \rho > 0. \quad (42)$$

If W is positive semidefinite, then

$$v^\top N v = v^\top A^\top W A v + \rho v^\top v \geq \rho \|v\|^2 \quad (43)$$

for all v . Hence N is positive definite and nonsingular. Without nonsingularity, the derivative of the solve may not be well defined.

7 The Fixed-Branch Forward Pass

At time t , the fixed branch constructs a nonnegative approximation

$$\hat{q}_t(z; \beta) = e^{-c_t} \phi_t(z; \beta)^2 + \tau_t \lambda_t(z) \quad (44)$$

and its normalizer

$$\hat{Z}_t(\beta) = \int \hat{q}_t(z; \beta) dz. \quad (45)$$

The branch is the collection of structural choices that define the scalar as a function of β . These choices include the domain, fitting points, basis, ranks, sweep order, shift, defensive term, and ridge parameter. The branch does not mean that the fitted numerical core values are held constant when β changes. Instead,

$$C_{t,k}(\beta) = \text{the core obtained by the saved fitting rule at parameter } \beta. \quad (45a)$$

The derivative pass computes $\dot{C}_{t,k}(\beta)$. This distinction matters: freezing the core values would usually make the normalizer nearly constant and would not test the derivative of the fitted approximation.

Forward object	What is stored	Why it is needed later
Domain map Ψ_t	Coordinate intervals and Jacobian	Evaluates the same transformed target at every derivative point
Fitting points $z^{(j)}$	Deterministic point array and weights	Defines the least-squares equations
Basis b_k	Basis family, degree, mass matrices	Evaluates cores and mass contractions
Ranks R_k	Fixed rank vector	Fixes core dimensions
Sweep order	Core update order and number of sweeps	Defines a finite composition of solve maps
Shift c_t	One frozen scalar	Defines the same scaled square-root target
Defensive term	τ_t, λ_t	Ensures nonnegative density floor
Cores $C_{t,k}(\beta)$	Parameter-dependent fitted coefficient arrays	Evaluate ϕ_t and $\hat{q}_t$
Mass contractions	$M_{t,>k}(\beta), M_{t,<k}(\beta)$	Compute integrals and marginals
Normalizer	$\hat{Z}_t$	Adds $\log \hat{Z}_t$ to the scalar
Carried filter	Numerator a_t and normalized $\hat{p}_t$	Feeds the next time step

The transformed target fitted at time t is

$$\tilde{q}_t(z; \beta) = \hat{p}_{t-1}(x_{t-1}(z); \beta) f_\beta(x_t(z) | x_{t-1}(z)) g_\beta(y_t | x_t(z)) J_t(z). \quad (46)$$

The fitted square-root values are

$$y_j(\beta) = e^{c_t/2} \sqrt{\tilde{q}_t(z^{(j)}; \beta)}. \quad (47)$$

The tensor train is

$$\phi_t(z; \beta) = H_{t,1}(z_1; \beta) H_{t,2}(z_2; \beta) \cdots H_{t,D}(z_D; \beta), \quad (48)$$

where $H_{t,k}(z_k) \in \mathbb{R}^{R_{k-1} \times R_k}$ and $R_0 = R_D = 1$.

The retained numerator for the next filter is

$$a_t(z_t; \beta) = \int \hat{q}_t(z_t, z_{t-1}; \beta) dz_{t-1}. \quad (49)$$

The retained normalized filter is

$$\hat{p}_t(z_t; \beta) = \frac{a_t(z_t; \beta)}{\hat{Z}_t(\beta)}. \quad (50)$$

If the filter is reported in physical coordinates, multiply by the appropriate change-of-variables factor. The derivative logic is unchanged if the domain map is frozen.

8 The Fixed-Branch Derivative Pass

The derivative pass computes the derivative of each forward object that depends on β . It does not create a new branch.

Derivative object	Forward object differentiated	Formula source
$\dot{\tilde{q}}_{t,j}$	target value at fitting point	product rule
$\dot{y}_j$	square-root fit value	chain rule
$\dot{A}_k, \dot{N}_k, \dot{d}_k$	least-squares system	environment derivatives
$\dot{C}_{t,k}$	fitted core	fixed linear solve
$\dot{M}_{t,>k}$	mass contraction	product rule in core indices
$\dot{\hat{Z}}_t$	normalizer	squared-integral derivative
$\dot{a}_t$	carried numerator	marginal contraction derivative
$\dot{\hat{p}}_t$	normalized carried filter	quotient rule
$\dot{\tilde{q}}_{t+1}$	next target	product rule using $\hat{\tilde{p}}_t$

8.1 Target And Square-Root Target

Differentiate (46). At fitting point $z^{(j)}$, abbreviate

$$\hat{p}_{t-1,j} = \hat{p}_{t-1}(x_{t-1}^{(j)}; \beta), \quad f_j = f_\beta(x_t^{(j)} | x_{t-1}^{(j)}), \quad g_j = g_\beta(y_t | x_t^{(j)}). \quad (51)$$

With $J_{t,j}$ frozen, the derivative is

$$\dot{\tilde{q}}_{t,j} = J_{t,j} \left[\dot{\hat{p}}_{t-1,j} f_j g_j + \hat{p}_{t-1,j} \dot{f}_j g_j + \hat{p}_{t-1,j} f_j \dot{g}_j \right]. \quad (52)$$

If the domain map depended on β , an additional $\hat{p} f g \dot{J}$ term would appear. That is outside the fixed branch used here.

From (47),

$$\dot{y}_j = e^{c_t/2} \frac{1}{2\sqrt{\tilde{q}_{t,j}}} \dot{\tilde{q}}_{t,j}, \quad (53)$$

provided $\tilde{q}_{t,j} > 0$. A numerical implementation should use a positive floor if the target is close to zero. If such a floor is used, the flooring rule becomes part of the declared scalar. For example,

$$y_j(\beta) = e^{c_t/2} \sqrt{\max\{\tilde{q}_{t,j}(\beta), \varepsilon_{\text{floor}}\}} \quad (53a)$$

has derivative zero through the floor whenever $\tilde{q}_{t,j} < \varepsilon_{\text{floor}}$. The formulas below describe the smooth positive-target case; a floored implementation must differentiate the floored scalar it actually evaluates.

8.2 Design Matrix And Core Solve

The design row is the first place where tensor-train notation can look more mysterious than it is. Fix one sample point $z^{(j)}$ and one core k . All cores except core k are treated as the known left and right environments for this local least-squares update. Write

$$L_{j,k}[a] = \left[H_1(z_1^{(j)}) \cdots H_{k-1}(z_{k-1}^{(j)}) \right]_{1a}, \quad (53b)$$

$$R_{j,k}[b] = \left[H_{k+1}(z_{k+1}^{(j)}) \cdots H_D(z_D^{(j)}) \right]_{b1}. \quad (53c)$$

The local core evaluated at $z_k^{(j)}$ is

$$H_k(z_k^{(j)})[a, b] = \sum_{\ell=1}^{m_k} C_k[a, \ell, b] b_k(z_k^{(j)})[\ell], \quad (53d)$$

where m_k is the number of one-dimensional basis functions in coordinate k . The value of the tensor train at this one point is therefore

$$\begin{aligned} \phi_t(z^{(j)}; \beta) &= \sum_{a=1}^{R_{k-1}} \sum_{b=1}^{R_k} L_{j,k}[a] H_k(z_k^{(j)})[a, b] R_{j,k}[b] \\ &= \sum_{a=1}^{R_{k-1}} \sum_{\ell=1}^{m_k} \sum_{b=1}^{R_k} L_{j,k}[a] b_k(z_k^{(j)})[\ell] R_{j,k}[b] C_k[a, \ell, b]. \end{aligned} \quad (53e)$$

Equation (53e) is the whole reason the local fit is a linear least-squares problem. At the fixed sample point, the unknown numbers are the entries of the core C_k . Every coefficient multiplying $C_k[a, \ell, b]$ is already known from the branch and from the other cores.

If $g_k = \text{vec}(C_k)$ stacks entries in the order (a, ℓ, b) , the row coefficient of $g_k[a, \ell, b]$ is

$$A_{j,k}[a, \ell, b] = L_{j,k}[a] b_k(z_k^{(j)})[\ell] R_{j,k}[b]. \quad (53f)$$

The compact Kronecker expression in (57) is just (53f) written without three sums:

$$A_{j,k} = R_{j,k}^\top \otimes b_k(z_k^{(j)})^\top \otimes L_{j,k}. \quad (53g)$$

To differentiate the row, keep the basis value fixed and differentiate the two environments:

$$\begin{aligned} \dot{A}_{j,k}[a, \ell, b] &= \dot{L}_{j,k}[a] b_k(z_k^{(j)})[\ell] R_{j,k}[b] \\ &\quad + L_{j,k}[a] b_k(z_k^{(j)})[\ell] \dot{R}_{j,k}[b]. \end{aligned} \quad (53h)$$

Equation (58) is (53h) written in the same Kronecker shorthand. If the basis or fitting point moved with β , there would be an additional term with $\dot{b}_k(z_k^{(j)})$. That term is absent because the branch keeps basis and fitting points fixed.

For one core k , the least-squares relation has rows

$$\phi_t(z^{(j)}; \beta) = A_{j,k}(\beta)g_k(\beta), \quad (54)$$

where $g_k = \text{vec}(C_{t,k})$. Let

$$L_{j,k} = H_1(z_1^{(j)}) \cdots H_{k-1}(z_{k-1}^{(j)}), \quad (55)$$

$$R_{j,k} = H_{k+1}(z_{k+1}^{(j)}) \cdots H_D(z_D^{(j)}). \quad (56)$$

Then

$$A_{j,k} = R_{j,k}^\top \otimes b_k(z_k^{(j)})^\top \otimes L_{j,k}. \quad (57)$$

Because basis values and fitting points are frozen,

$$\dot{A}_{j,k} = \dot{R}_{j,k}^\top \otimes b_k(z_k^{(j)})^\top \otimes L_{j,k} + R_{j,k}^\top \otimes b_k(z_k^{(j)})^\top \otimes \dot{L}_{j,k}. \quad (58)$$

The environments are differentiated by product rule:

$$\dot{L}_{j,1} = 0, \quad \dot{L}_{j,k+1} = \dot{L}_{j,k}H_k(z_k^{(j)}) + L_{j,k}\dot{H}_k(z_k^{(j)}), \quad (59)$$

$$\dot{R}_{j,D} = 0, \quad \dot{R}_{j,k-1} = \dot{H}_k(z_k^{(j)})R_{j,k} + H_k(z_k^{(j)})\dot{R}_{j,k}. \quad (60)$$

The local core matrix derivative is

$$\dot{H}_k(z_k^{(j)})[a, b] = \sum_{\ell} \dot{C}_k[a, \ell, b] b_k(z_k^{(j)})[\ell]. \quad (61)$$

The fixed ridge solve is

$$N_k g_k = d_k, \quad N_k = A_k^\top W A_k + \rho I, \quad d_k = A_k^\top W y. \quad (62)$$

Using (40),

$$N_k \dot{g}_k = \dot{d}_k - \dot{N}_k g_k, \quad (63)$$

where

$$\dot{N}_k = \dot{A}_k^\top W A_k + A_k^\top W \dot{A}_k, \quad (64)$$

$$\dot{d}_k = \dot{A}_k^\top W y + A_k^\top W \dot{y}. \quad (65)$$

The derivative update is therefore another linear solve with the same left-hand matrix N_k . This is why the fixed branch is implementable: the forward pass and derivative pass use the same stored computational path.

8.3 Mass Contraction And Normalizer

The full right-mass recursion is the same idea as the rank- R warmup in (29)–(36). In the warmup, $M_1[r, s]$ stored the inner products of the first-coordinate factors and $M_2[r, s]$ stored the inner products of the second-coordinate factors. In the full tensor train, $M_{>j}[b, b']$ plays the role of the already-integrated right factor, $B_j[\ell, \ell']$ is the one-coordinate basis inner product, and the two copies of C_j are the two factors coming from squaring ϕ_t .

The right mass contraction is

$$M_{>j-1}[a, a'] = \sum_{b, b', \ell, \ell'} C_j[a, \ell, b] M_{>j}[b, b'] C_j[a', \ell', b'] B_j[\ell, \ell']. \quad (66)$$

Here B_j is the basis mass matrix. It is frozen. Differentiate (66):

$$\begin{aligned} \dot{M}_{>j-1}[a, a'] &= \sum_{b, b', \ell, \ell'} \dot{C}_j[a, \ell, b] M_{>j}[b, b'] C_j[a', \ell', b'] B_j[\ell, \ell'] \\ &\quad + \sum_{b, b', \ell, \ell'} C_j[a, \ell, b] \dot{M}_{>j}[b, b'] C_j[a', \ell', b'] B_j[\ell, \ell'] \\ &\quad + \sum_{b, b', \ell, \ell'} C_j[a, \ell, b] M_{>j}[b, b'] \dot{C}_j[a', \ell', b'] B_j[\ell, \ell']. \end{aligned} \quad (67)$$

Start from

$$M_{>D} = 1, \quad \dot{M}_{>D} = 0. \quad (68)$$

Then (66)–(67) move from the right end of the chain to the left end. The final square integral is

$$R_t(\beta) = \int \phi_t(z; \beta)^2 dz = M_{>0}. \quad (69)$$

Its derivative is

$$\dot{R}_t(\beta) = \dot{M}_{>0}. \quad (70)$$

By (17)–(18),

$$\widehat{Z}_t(\beta) = e^{-c_t} R_t(\beta) + \tau_t, \quad (71)$$

$$\dot{\widehat{Z}}_t(\beta) = e^{-c_t} \dot{R}_t(\beta). \quad (72)$$

8.4 Carried Filter And Next Time Step

The carried numerator is

$$a_t(z_t; \beta) = \int \widehat{q}_t(z_t, z_{t-1}; \beta) dz_{t-1}. \quad (73)$$

Its derivative is

$$\dot{a}_t(z_t; \beta) = \int \dot{\widehat{q}}_t(z_t, z_{t-1}; \beta) dz_{t-1}. \quad (74)$$

For the squared part of the approximation, this marginal is another tensor contraction. To make the formula concrete, suppose the retained coordinate is the last coordinate $z_D = z_t$, and the previous-state coordinates are $z_1, \dots, z_{D-1}$. First integrate out the previous-state coordinates by the same left-to-right mass recursion:

$$M_{<0} = 1, \quad M_{<j}[b, b'] = \sum_{a, a', \ell, \ell'} M_{<j-1}[a, a'] C_j[a, \ell, b] C_j[a', \ell', b'] B_j[\ell, \ell'] \quad (74a)$$

for $j = 1, \dots, D - 1$. The derivative of this marginal environment is

$$\begin{aligned}\dot{M}_{<j}[b, b'] &= \sum_{a, a', \ell, \ell'} \dot{M}_{<j-1}[a, a'] C_j[a, \ell, b] C_j[a', \ell', b'] B_j[\ell, \ell'] \\ &+ \sum_{a, a', \ell, \ell'} M_{<j-1}[a, a'] \dot{C}_j[a, \ell, b] C_j[a', \ell', b'] B_j[\ell, \ell'] \\ &+ \sum_{a, a', \ell, \ell'} M_{<j-1}[a, a'] C_j[a, \ell, b] \dot{C}_j[a', \ell', b'] B_j[\ell, \ell'].\end{aligned}\tag{74b}$$

Start with $\dot{M}_{<0} = 0$. After the recursion reaches $D - 1$, the retained-coordinate numerator is the function

$$\begin{aligned}a_t(z_D; \beta) &= e^{-c_t} \sum_{b, b', \ell, \ell'} M_{<D-1}[b, b'] C_D[b, \ell, 1] C_D[b', \ell', 1] b_D(z_D)[\ell] b_D(z_D)[\ell'] \\ &+ \tau_t \int \lambda_t(z_D, z_{1:D-1}) dz_{1:D-1}.\end{aligned}\tag{74c}$$

Its derivative, with c_t, τ_t, λ_t and the basis fixed, is

$$\begin{aligned}\dot{a}_t(z_D; \beta) &= e^{-c_t} \sum_{b, b', \ell, \ell'} \dot{M}_{<D-1}[b, b'] C_D[b, \ell, 1] C_D[b', \ell', 1] b_D(z_D)[\ell] b_D(z_D)[\ell'] \\ &+ e^{-c_t} \sum_{b, b', \ell, \ell'} M_{<D-1}[b, b'] \dot{C}_D[b, \ell, 1] C_D[b', \ell', 1] b_D(z_D)[\ell] b_D(z_D)[\ell'] \\ &+ e^{-c_t} \sum_{b, b', \ell, \ell'} M_{<D-1}[b, b'] C_D[b, \ell, 1] \dot{C}_D[b', \ell', 1] b_D(z_D)[\ell] b_D(z_D)[\ell'].\end{aligned}\tag{74d}$$

Equations (74a)–(74d) are the coding recipe behind the short integral identity (74). If the retained state is not the last coordinate, reorder the coordinates or use left and right environments around the retained block; the same product-rule pattern applies.

The normalized carried filter is

$$\hat{p}_t(z_t; \beta) = \frac{a_t(z_t; \beta)}{\hat{Z}_t(\beta)}.\tag{75}$$

Differentiate by quotient rule:

$$\dot{\hat{p}}_t(z_t; \beta) = \frac{\dot{a}_t(z_t; \beta)}{\hat{Z}_t(\beta)} - \frac{a_t(z_t; \beta) \dot{\hat{Z}}_t(\beta)}{\hat{Z}_t(\beta)^2}.\tag{76}$$

This is the crucial bridge between time steps. At time $t + 1$, the target contains $\hat{p}_t$, so its derivative contains $\dot{\hat{p}}_t$. Without saving the carried-filter derivative, the next-step gradient is incomplete.

9 Proposition 1: The Fixed-Branch Recursion Is A Normalized Approximate Filter

Proposition 1. *Fix a parameter value β , a time horizon T , and fixed branch objects for all times. Suppose that each approximate joint density*

$$\hat{q}_t(z_t, z_{t-1}; \beta) = e^{-c_t} \phi_t(z_t, z_{t-1}; \beta)^2 + \tau_t \lambda_t(z_t, z_{t-1})\tag{77}$$

is nonnegative and integrable, and that

$$0 < \widehat{Z}_t(\beta) = \int \widehat{q}_t(z_t, z_{t-1}; \beta) dz_t dz_{t-1} < \infty. \quad (78)$$

Define

$$\widehat{p}_t(z_t; \beta) = \frac{\int \widehat{q}_t(z_t, z_{t-1}; \beta) dz_{t-1}}{\widehat{Z}_t(\beta)}. \quad (79)$$

Then $\widehat{p}_t(\cdot; \beta)$ is a normalized density in z_t . The recursion is a Bayesian filtering recursion for the declared approximate joint density $\widehat{q}_t$, not a proof that $\widehat{q}_t$ equals the exact Bayesian joint density.

Proof. Nonnegativity follows immediately:

$$e^{-c_t} \phi_t^2 \geq 0, \quad \tau_t \lambda_t \geq 0. \quad (80)$$

Therefore $\widehat{q}_t \geq 0$. By assumption, $\widehat{Z}_t$ is positive and finite, so

$$\widehat{p}_t^{\text{joint}}(z_t, z_{t-1}; \beta) = \frac{\widehat{q}_t(z_t, z_{t-1}; \beta)}{\widehat{Z}_t(\beta)} \quad (81)$$

is a normalized joint density:

$$\begin{aligned} \iint \widehat{p}_t^{\text{joint}}(z_t, z_{t-1}; \beta) dz_t dz_{t-1} &= \frac{1}{\widehat{Z}_t(\beta)} \iint \widehat{q}_t(z_t, z_{t-1}; \beta) dz_t dz_{t-1} \\ &= 1. \end{aligned} \quad (82)$$

The retained filter is the marginal of this normalized joint density:

$$\widehat{p}_t(z_t; \beta) = \int \widehat{p}_t^{\text{joint}}(z_t, z_{t-1}; \beta) dz_{t-1}. \quad (83)$$

Integrating (83) over z_t ,

$$\begin{aligned} \int \widehat{p}_t(z_t; \beta) dz_t &= \int \left[\int \widehat{p}_t^{\text{joint}}(z_t, z_{t-1}; \beta) dz_{t-1} \right] dz_t \\ &= \iint \widehat{p}_t^{\text{joint}}(z_t, z_{t-1}; \beta) dz_t dz_{t-1} \\ &= 1. \end{aligned} \quad (84)$$

Thus $\widehat{p}_t$ is normalized. The proof used only the declared approximation $\widehat{q}_t$. It did not use the exact Bayesian density except as motivation for constructing $\widehat{q}_t$. Hence the theorem is a normalization theorem for the approximate filtering recursion. $\square$

10 Proposition 2: The Fixed-Branch Derivative Differentiates The Same Scalar

Proposition 2. Fix the branch $B = (B_1, \dots, B_T)$. Assume:

$$\widehat{\ell}_T(\beta; B) = \sum_{t=1}^T \log \widehat{Z}_t(\beta; B_t), \quad (85)$$

each $\hat{Z}_t$ is positive and finite, all model density evaluations are differentiable in β , every least-squares update uses a fixed nonsingular ridge system, and differentiation under the finite-dimensional contractions and integrals is valid. Then the derivative pass defined by (52)–(76) computes

$$\frac{\partial}{\partial \beta} \hat{\ell}_T(\beta; B). \quad (86)$$

Proof. The proof follows the computational graph of the fixed branch.

First, the transformed target (46) is a product of the previous carried filter, transition density, observation density, and frozen Jacobian. The product rule therefore gives (52). If $t = 1$, the previous filter is the initial density; for $t > 1$, its derivative is supplied by the previous carried-filter derivative (76).

Second, the fitted target values (47) are differentiable functions of $\tilde{q}_{t,j}$. The chain rule gives (53). The stabilizing shift c_t is frozen, so no $\dot{c}_t$ term appears.

Third, each core update is a fixed linear solve. Equations (37)–(41) prove that differentiating the solve yields

$$N_k \dot{g}_k = \dot{d}_k - \dot{N}_k g_k. \quad (87)$$

Equations (54)–(65) identify $N_k, d_k, \dot{N}_k, \dot{d}_k$ for the fixed least-squares update. Since the number and order of solves are fixed, a finite composition of differentiable solve maps gives differentiable fitted cores.

Fourth, the square integral $R_t = \int \phi_t^2$ is computed by the mass recursion (66). Its derivative is the product-rule recursion (67). Therefore

$$\dot{R}_t = \frac{\partial}{\partial \beta} \int \phi_t(z; \beta)^2 dz. \quad (88)$$

With frozen c_t, τ_t, λ_t , (71)–(72) give the derivative of $\hat{Z}_t$.

Fifth, the carried numerator and carried filter are defined by (73)–(75). Their derivatives are (74) and (76), obtained by differentiating under the integral and by the quotient rule. This supplies the derivative object needed by the next time step.

Finally, since each time contribution is $\log \hat{Z}_t(\beta; B_t)$, Warmup 1 gives

$$\frac{\partial}{\partial \beta} \log \hat{Z}_t(\beta; B_t) = \frac{\dot{\hat{Z}}_t(\beta; B_t)}{\hat{Z}_t(\beta; B_t)}. \quad (89)$$

Summing (89) over $t = 1, \dots, T$ gives (86).

Every object differentiated in this proof is an object in the saved branch B . The proof never differentiates a changed rank, changed grid, changed shift, changed pivot set, or changed domain. Therefore the derivative is the derivative of the same scalar $\hat{\ell}_T(\beta; B)$, not the derivative of an adaptive algorithm that chooses a new B when β changes. $\square$

11 What This Does Not Prove

The result above is deliberately narrow.

It does not prove exact posterior accuracy:

$$\hat{q}_t = q_t \quad \text{is not claimed.} \quad (90)$$

It proves normalization and differentiation of the declared approximation.

It does not prove ranks stay small:

$$R_k \text{ moderate for all } k \text{ is a diagnostic requirement, not a theorem here.} \quad (91)$$

It does not differentiate adaptive decisions:

$$\frac{\partial}{\partial \beta} \{\text{chosen ranks, pivots, domains, shifts, fitting points}\} \text{ is not part of the formula.} \quad (92)$$

It does not prove HMC convergence:

$$\text{correct local gradient of an approximate scalar} \neq \text{posterior sampling guarantee.} \quad (93)$$

These caveats make the result more honest, not weaker. The fixed-branch gradient is a precise mathematical contract. Empirical accuracy, rank stability, and sampling performance must be tested separately.

12 Finite-Difference Test

The finite-difference test checks that the value path and derivative path agree for the same branch. Choose a base parameter β_0 . Build the branch once and save the structural choices

$$B = \{\text{domains, points, basis, ranks, sweeps, shifts,} \\ \text{defensive terms, ridge parameters,} \\ \text{deterministic initialization rules}\}. \quad (94)$$

Then evaluate the same scalar at perturbed parameters while reusing those structural choices and recomputing the parameter-dependent fitted values by the same fixed equations:

$$\widehat{\ell}_T(\beta_0 + he_i; B), \quad \widehat{\ell}_T(\beta_0 - he_i; B). \quad (95)$$

That means the domains, points, ranks, shifts, and sweep order are unchanged, but the fitted core values

$$C_{t,k}(\beta_0 + he_i; B) \quad \text{and} \quad C_{t,k}(\beta_0 - he_i; B) \quad (95a)$$

are recomputed from the same saved fitting rule. They are not copied from β_0 . The centered finite difference is

$$D_i(h) = \frac{\widehat{\ell}_T(\beta_0 + he_i; B) - \widehat{\ell}_T(\beta_0 - he_i; B)}{2h}. \quad (96)$$

Compare it to the analytical derivative

$$G_i = \partial_i \widehat{\ell}_T(\beta_0; B). \quad (97)$$

Record

$$e_i(h) = |D_i(h) - G_i|, \quad (98)$$

and

$$e_i^{\text{rel}}(h) = \frac{|D_i(h) - G_i|}{1 + |G_i|}. \quad (99)$$

Use a small schedule such as

$$h \in \{10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}\}. \quad (100)$$

A healthy derivative check typically shows decreasing error over a range of h , followed by roundoff or fitting-noise limitations. If the error does not decrease on any reasonable range, the derivative path is not yet aligned with the value path.

The branch identity check is mandatory:

$$B(\beta_0) = B(\beta_0 + he_i) = B(\beta_0 - he_i) = B. \quad (101)$$

If the implementation rebuilds ranks, points, shifts, or domains at the perturbed values, then (96) is no longer testing the derivative of $\widehat{\ell}_T(\beta; B)$. It is testing a different adaptive calculation. If the implementation copies the numerical cores from β_0 without resolving the fixed fitting equations at the perturbed parameter, then it is also testing the wrong object. The correct test freezes the branch choices but allows all differentiable outputs of those choices to change with β .

13 Minimal Runnable Example

The smallest useful example should be nonlinear but one-dimensional:

$$x_t = \beta x_{t-1} + \sigma \epsilon_t, \quad \epsilon_t \sim N(0, 1), \quad (102)$$

$$y_t = \sin(x_t) + \eta_t, \quad \eta_t \sim N(0, r^2). \quad (103)$$

Use $T = 2$. The first time step builds

$$q_1(x_1, x_0; \beta) = p_\beta(x_0) f_\beta(x_1 | x_0) g_\beta(y_1 | x_1). \quad (104)$$

It fits ϕ_1 , computes

$$\widehat{Z}_1(\beta) = e^{-c_1} \int \phi_1(z_1, z_0; \beta)^2 dz_1 dz_0 + \tau_1, \quad (105)$$

and stores

$$\widehat{p}_1(z_1; \beta) = \frac{\int [e^{-c_1} \phi_1(z_1, z_0; \beta)^2 + \tau_1 \lambda_1(z_1, z_0)] dz_0}{\widehat{Z}_1(\beta)}. \quad (106)$$

The second time step must call that stored object:

$$q_2(x_2, x_1; \beta) = \widehat{p}_1(x_1; \beta) f_\beta(x_2 | x_1) g_\beta(y_2 | x_2). \quad (107)$$

The scalar printed by the example is

$$\widehat{\ell}_2(\beta) = \log \widehat{Z}_1(\beta) + \log \widehat{Z}_2(\beta). \quad (108)$$

The script should print

$$\widehat{\ell}_2(\beta_0), \quad \partial_\beta \widehat{\ell}_2(\beta_0), \quad D(h), \quad |D(h) - \partial_\beta \widehat{\ell}_2(\beta_0)|. \quad (109)$$

It should also print the branch manifest or manifest hash used for β_0 , $\beta_0 + h$, and $\beta_0 - h$. The test passes only if those branch identifiers are identical and the derivative error has a stable decreasing window before numerical roundoff dominates.

14 Closing Summary

The fixed-branch gradient is built from five elementary ideas:

normalizer derivative	$\partial \log Z = \dot{Z}/Z,$	
squared density derivative	$\partial(\phi^2) = 2\phi\dot{\phi},$	
mass contraction derivative	differentiate every factor once,	(110)
linear solve derivative	$N\dot{g} = \dot{d} - \dot{N}g,$	
carried filter derivative	$\partial(a/Z) = \dot{a}/Z - a\dot{Z}/Z^2.$	

The tensor-train notation makes these ideas look large, but it does not change their nature. Once the branch is fixed, the forward pass defines one scalar. The derivative pass differentiates that scalar. What remains for empirical work is to test whether the approximation is accurate enough, stable enough, and useful enough for the target scientific model.